How the complex sort works, step by step

Two-pivot partition, then fixed_sort and cheapest insertion back: algorithms/complex.c, reusing fixed_sort.c and the simple-sort helpers

The idea: like quick sort, split A into three bands by rank with two pivots: small to the bottom of B, middle to the top of B, big stays in A. The pivots slide up after every push until 5 numbers are left, which `fixed_sort` sorts. Then B comes back with the same cheapest-insertion pass as the simple and medium sorts.

1 Pivots

Before every move, split the ranks with two pivots: p1 = size_b + size_a x 0.35, p2 = size_b + size_a x 0.65.

`find_pivot`

2 Partition

Top of A below p1: pb then rb. Between p1 and p2: pb. p2 or above: ra. Stop when 5 are left in A.

`push_to_b`

3 Last five

Sort the 5 numbers left in A: push the two smallest, sort three, push them back.

`fixed_sort`

4 Back to A

While B is not empty: push the cheapest into A, on top of its next-bigger number.

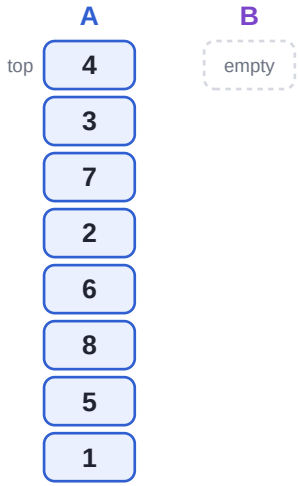
`get_cheapest_move(to_b = 0)`

5 Align

A is sorted but maybe rotated. Rotate its minimum to the top the short way.

`final_alignment`

THE EXAMPLE IN THIS WALKTHROUGH



Input `./push_swap --complex 4 3 7 2 6 8 5 1`. The first argument ends up on top of A. It is the same input as the simple and medium walkthroughs.

The code compares each node's `index` (its rank), not its value. With the numbers 1 to 8 the rank is just value minus 1, so the pages show values; when a rule talks about ranks, it says so.

In total the algorithm needs **19 operations**. Every step on the following pages was produced by a Python replica of the C code whose output was checked against the real `push_swap` binary.

Colours: an orange box is the number chosen to move, a teal box is its target (the number it will land on).

The rules behind the partition

Where each number goes, and how the pivots move.

1. Three bands, by rank

`find_pivot` computes two cut points from the current sizes. They are stored in an `int`, so the fractional part is dropped. Then the top of A goes into one of three bands:

- rank < p1: **pb** then **rb** (bottom of B)
- p1 <= rank < p2: **pb** only (top of B)
- rank >= p2: **ra**, stays in A for now

Shown for the first step of the example: 8 numbers, p1 = 2, p2 = 5.

2. The pivots slide up after every push

size_a + size_b is always n, so every pb raises p1 by 0.65 and p2 by 0.35 (before rounding). An ra changes neither size, so the pivots stay put. Big numbers skipped earlier fall below p2 later and get pushed on a later lap around A: the same three-way split, repeated on whatever is left.

size A	size B	p1 = size_b + size_a x 0.35	p2 = size_b + size_a x 0.65
8	0	0 + 2.8 = 2.8 -> 2	0 + 5.2 = 5.2 -> 5
7	1	1 + 2.45 = 3.45 -> 3	1 + 4.55 = 5.55 -> 5
6	2	2 + 2.1 = 4.1 -> 4	2 + 3.9 = 5.9 -> 5

Why it always finds something to push

Every number in B had rank < p2 when it was pushed, and p2 never goes down, so all of B sits below the current p2. There are p2 ranks below p2 and B holds size_b of them, so A still holds p2 - size_b = int(size_a x 0.65) of them. While size_a > 5 that is at least 3, so the ra loop always reaches a pushable number.

The shape of B

Low numbers are rotated to the bottom, middle numbers stay on top. As the pivots rise, each new middle band sits above the last one, so B is roughly sorted with the biggest pushed numbers near the top. Phase 4 does not rely on that order (it prices every number), it just finds cheaper moves.

Two small details

`lst = stacks->a` is saved before `pb`. After the push that same node is the top of B, so `lst->index < p1` tests the number that was just pushed.

The rb is skipped while B holds a single number (`ft_lstsize(stacks->b) >= 2`): rotating one number changes nothing and would only waste an operation.

The first split, by input size

n	p1	p2	low / middle / stays in A
100	35	65	ranks 0..34 / 35..64 / 65..99
500	175	325	ranks 0..174 / 175..324 / 325..499

Partition: steps 1 to 4

p1 to p2, top of B | below p1, bottom of B | p2 and up | already in B | top of A

Each card: the bands before the step, the decision, the operations, then A and B after it.

Step 1

p1 2 p2 5

A 8, B 0. Top of A is **4** (rank 3). 2 <= 3 < 5: push it, it stays on top.

`pb`

A	B
3	4
7	
2	
6	
8	
5	
1	

Step 2

p1 3 p2 5

A 7, B 1. Top of A is **3** (rank 2). 2 < p1 = 3: push it, then rb to the bottom.

`pb` `rb`

A	B
7	4
2	3
6	
8	
5	
1	

Step 3

p1 4 p2 5

A 6, B 2. Top of A is **7** (rank 6). 6 >= p2 = 5: ra and look again.

`ra`

A	B
2	4
6	3
8	
5	
1	
7	

Step 4

p1 4 p2 5

A 6, B 2. Top of A is **2** (rank 1). 1 < p1 = 4: push it, then rb to the bottom.

`pb` `rb`

A	B
6	4
8	3
5	2
1	
7	

Where B ended up

A has 5 numbers, so the loop stops. Phase total: **6 ops** (3 pb, 2 rb, 1 ra).

B, top to bottom: 4 3 2. The middle-band 4 stayed on top, the low 3 and 2 went underneath.

A: 6 8 5 1 7. These are not the five biggest: 1 and 5 were never reached because the loop stops as soon as A is down to 5. fixed_sort sorts any 5 numbers.

With 100 or 500 numbers the loop runs many laps around A, and B ends up as a stack of thin, roughly ordered bands.

Sort the last five with fixed_sort

number being moved

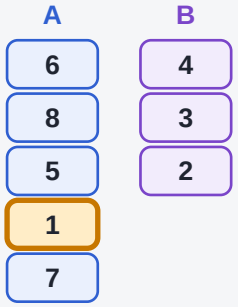
sort_five: push the two smallest to B, sort the remaining three, push the two back.

Push minimum 1

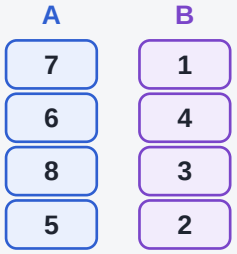
The minimum of A is **1**, at position 3 of 5. 3 > 5 / 2 = 2, so 2x rra brings it up, then pb.

```
rra
rra
pb
```

BEFORE



AFTER

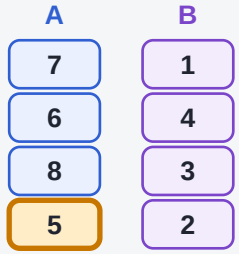


Push minimum 2

The minimum of A is **5**, at position 3 of 4. 3 > 4 / 2 = 2, so 1x rra brings it up, then pb.

```
rra
pb
```

BEFORE



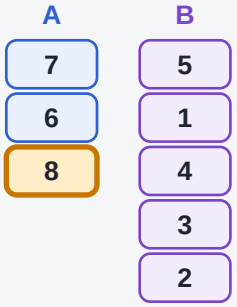
AFTER

Sort three

The max **8** is already at the bottom: no rotation. The top two are out of order: sa.

```
sa
```

BEFORE



AFTER

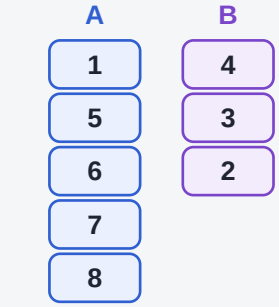
Bring them back

2x pa. The smallest comes back last, so A is fully sorted. B is back to what phase 2 left.

```
pa
pa
```

BEFORE

AFTER



Borrowing B

sort_five uses B as scratch space while it still holds the numbers from phase 2. The minimums are pushed on top and are the first ones popped, so the numbers underneath are never touched.

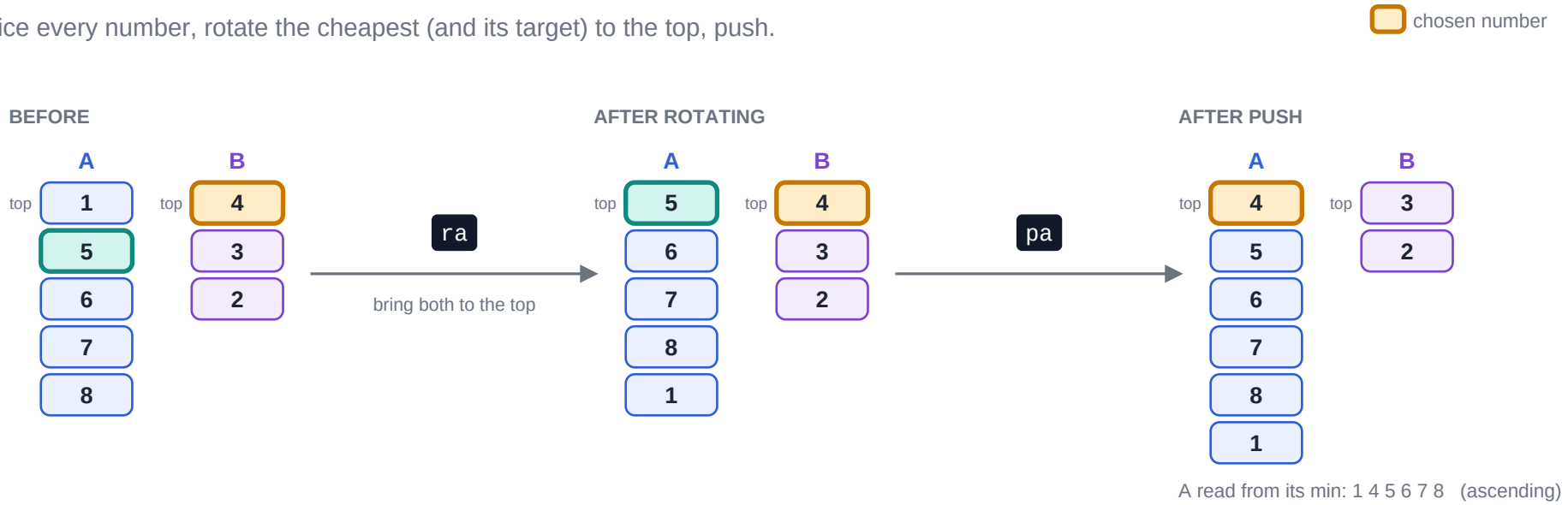
Each minimum is rotated up the short way: position <= size / 2 uses ra, otherwise rra.

Phase total: **8 ops**.

A is now 1 5 6 7 8, fully sorted, which is the base for phase 4.

Push 1 of 3: move 4 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
4	5	+1 (1 ra)	0	1
3	5	+1 (1 ra)	+1 (1 rb)	1
2	5	+1 (1 ra)	-1 (1 rrb)	2

Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

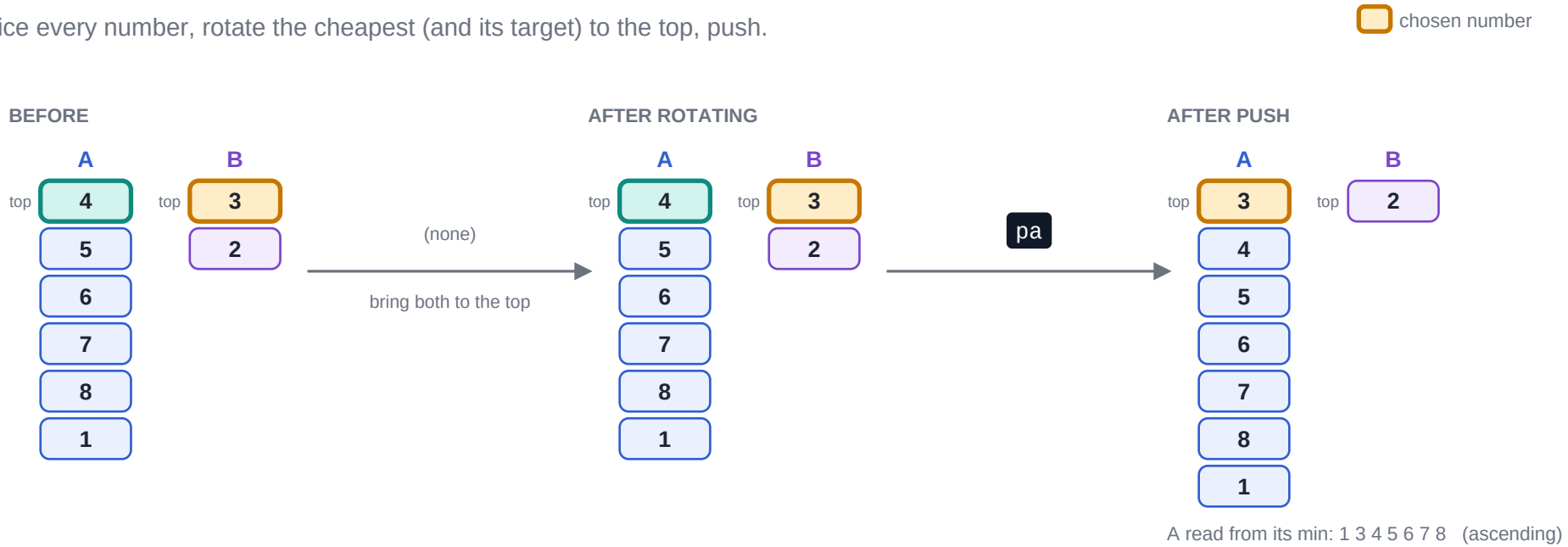
Cheapest: 4 (total 1; the first lowest total wins ties). The smallest number in A above 4 is 5. 3 also costs 1, but 4 comes first in B and the comparison is a strict <.

Moves: ra brings 5 to the top of A while 4 is already on top of B, then pa drops 4 onto 5.

This pass is exactly the one from the simple and medium sorts: `push_cheapest(stacks, 0, print)`. Same-sign costs share rr / rrr, opposite signs add up.

Push 2 of 3: move 3 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
3	4	0	0	0
2	4	0	+1 (1 rb)	1

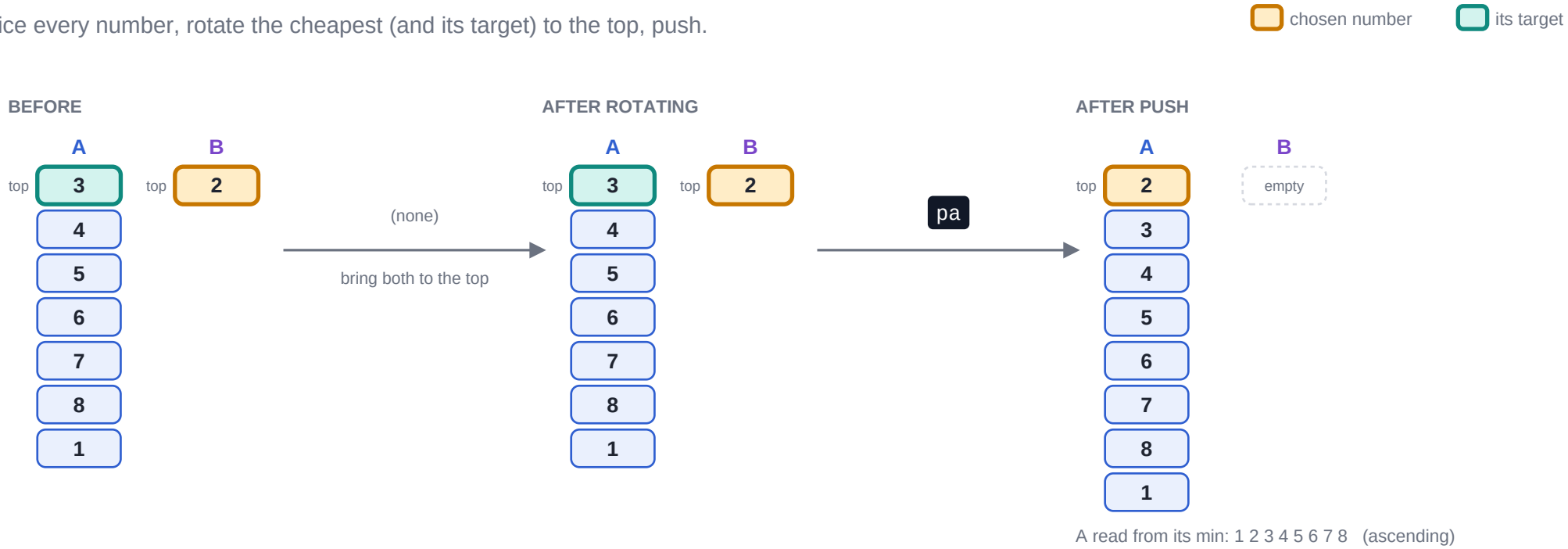
Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 3 (total 0; the first lowest total wins ties). The smallest number in A above 3 is 4.

Moves: 3 and 4 are both already on top, so no rotations: pa drops 3 straight onto 4.

Push 3 of 3: move 2 from B back to A

Price every number, rotate the cheapest (and its target) to the top, push.



EVERY CANDIDATE IS PRICED

number in B	target in A	cost A	cost B	total
2	3	0	0	0

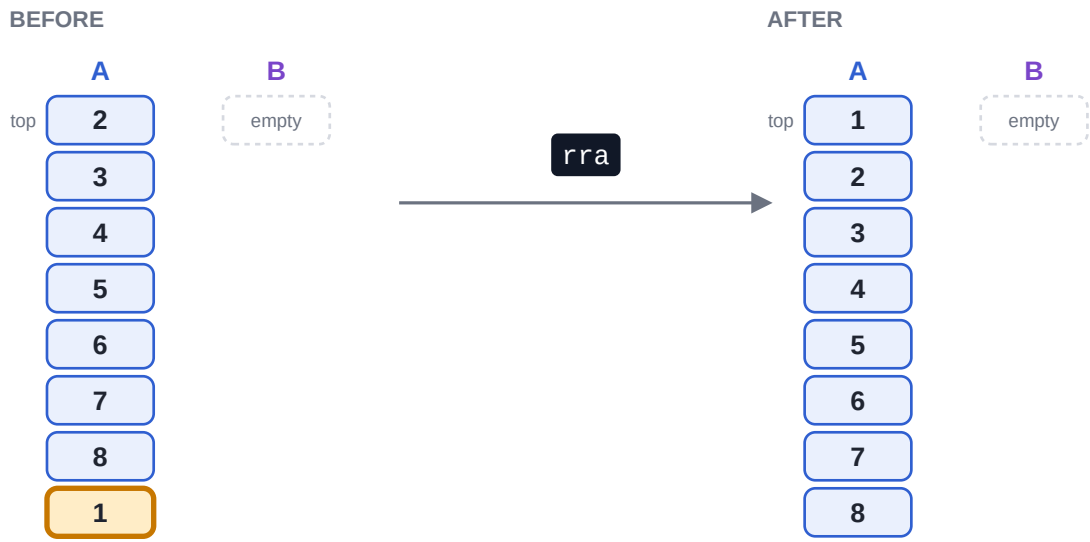
Target rule (B to A): the smallest number in A that is *bigger*. A is kept in ascending order, so the new number must sit right on top of its next-bigger neighbour.

Cheapest: 2 (total 0; the first lowest total wins ties). The smallest number in A above 2 is 3.

Moves: 2 and 3 are both already on top, so no rotations: pa drops 2 straight onto 3.

Rotate the minimum to the top

A is sorted in a circle but may start in the wrong place.



A reads 1 2 3 4 5 6 7 8 from its minimum, so it is sorted but rotated. The minimum 1 is at position 7 of 8.

`final_alignment` reuses the same signed cost: position 7 > size / 2 = 4, so the cost is 7 - 8 = -1, which means 1x `rra`.

Going forward would have taken 7x ra. One rra wraps the bottom number round to the top.

Done: sorted in 19 operations

The full operation list, exactly as ./push_swap --complex prints it.

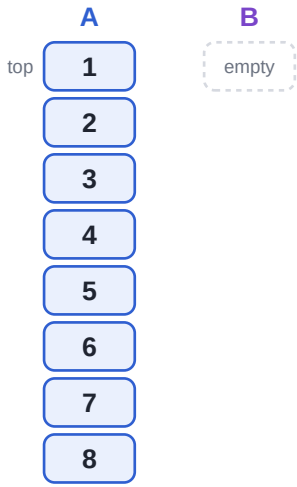
Partition
6 ops
`pb` `pb` `rb` `ra` `pb` `rb`

Last five
8 ops
`rra` `rra` `pb` `rra` `pb` `sa` `pa` `pa`

Back to A
4 ops
`ra` `pa` `pa` `pa`

Align
1 ops
`rra`

FINAL STACKS



Why only 19? On this input simple needs 16 and medium 30. With 8 numbers only 3 go through the partition and fixed_sort does most of the work; the partition only starts to pay off on large inputs.

average of 20 random inputs	simple	medium	complex
100 numbers	570	613	596
500 numbers	5134	4768	4675

Measured with the real binary on the same 20 inputs per size. Complex is ahead of medium at both sizes, still behind simple at 100 and ahead of both at 500. Worst case seen: 629 for 100, 4911 for 500.

Is it really quick sort? It borrows the partition idea (split by pivots, keep working on one part), but there is no recursion: the pivots slide up after every push, so the loop keeps re-partitioning what is left in A until 5 remain. That partition idea is why `sort.c` labels this strategy O(n log n).

Cost of the search: the partition is cheap, but phase 4 still prices every number in B and walks A to find its target for each push, so that part does $O(n^2)$ work, the same as the simple and medium sorts.